Ca' Foscari University of Venice

Development Methodology

Matteo Zanioli

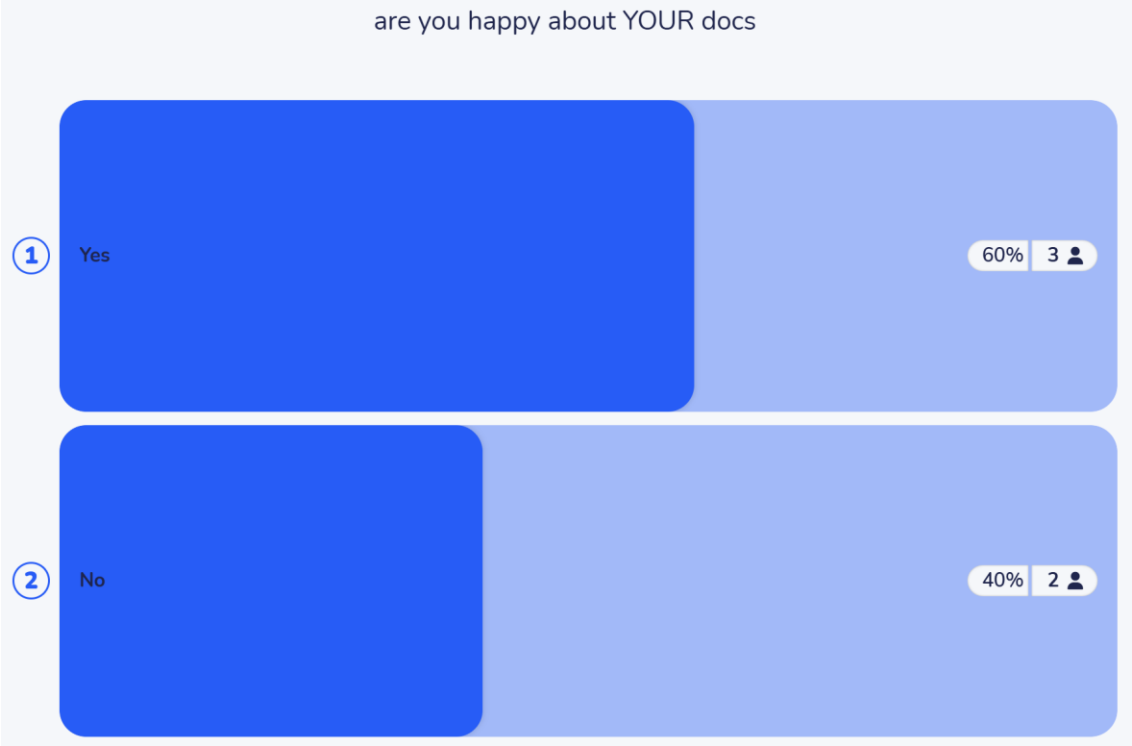
matteo.zanioli@unive.it

Ca' Foscari University of Venice

Documentation and how to write it

Ca' Foscari University of Venice

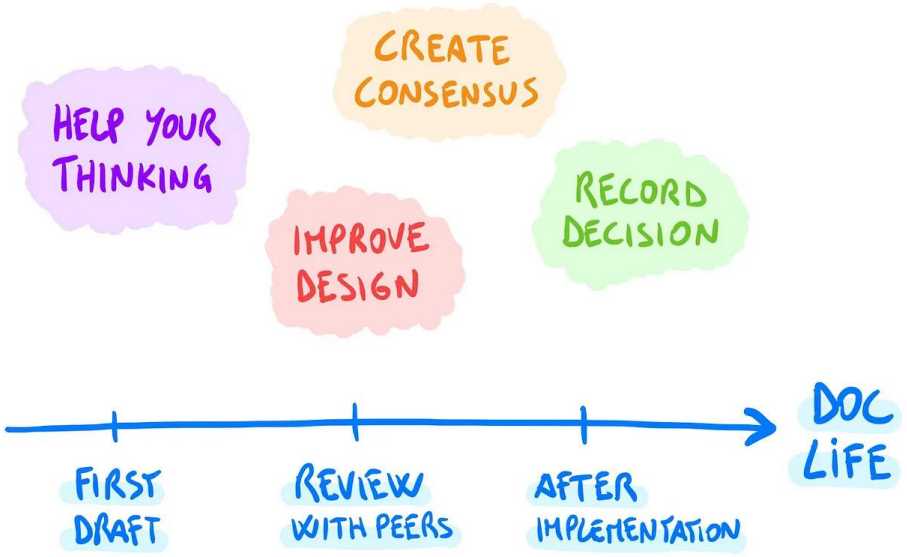
Are you happy about docs?



Ca' Foscari University of Venice

Are you happy about docs?

- Generally, nobody is satisfied about their documentation
  - people struggle at writing docs
  - confusion about what those are useful for and how to use them



- Different benefit
  - Help your reasoning, writing is thinking. Going through the process of writing a formal document helps solidify your reasoning and come up with a better solution.
  - Improve output, you can support your writing with previously created templates, checklists, and examples. These make the work lighter and lead to better results.
  - Improve communication, it is easier to have good conversations about complex topics when these are backed by writing. You get more thorough reasoning and easier convergence.
  - Future reference, docs naturally record decisions and can be used for posterity.
  - Onboarding support, documentation helps new team members quickly understand the software's functionality and the conventions used in the project

source: https://refactoring.fm/p/how-to-write-documentation

Writing docs

"Writing docs belongs to that category of dev tasks that everybody knows they should do, but nobody wants to do anyway.

It's an exclusive club with other illustrious members, such as writing tests, estimating tasks, or updating JIRA.

The natural restraint towards such tasks is often dismissed as laziness. But laziness carries meaning."

source: https://refactoring.fm/p/design-docs

Timing

- Writing docs only after the fact is like retrofitting unit tests to production code: not completely useless, but you miss out on a lot of value.
- **Writing docs** at the end is genuinely harder than doing it gradually **while the work is in progress**. Artifacts like meeting notes and design docs are not only useful per se — they naturally turn into long-term docs, later, with very little effort.
- Minimizing such effort is crucial to helping people keep up with docs. In fact, people are only going to create docs if the value from their use is perceived as superior to the writing effort.

source: https://refactoring.fm/p/how-to-write-documentation

Vicious or virtuous cycle

THE VICIOUS CYCLE OF DOCS

THE VIRTUOUS CYCLE OF DOCS

source: https://refactoring.fm/p/how-to-write-documentation

Type of documentation

- Technical documentation
  - Code documentation: comments within the code and documentation files that explain how functions, calls and modules work
  - API documentation: describes how to interact with the system's API, detailing endpoints, methods, parameter, and response
  - Architecture documentation: Diagrams and descriptions that explain the system's structure, dependencies between components, and how different parts of the software integrate.
- User documentation
  - User manuals: Detailed guides on using the software, aimed at end users. They may include tutorials, step-by-step guides, and FAQs.
  - Quick start guides: More concise versions of manuals, designed to help users get started quickly.
  - Inline documentation: Explanations directly integrated into the software's user interface, such as tooltips or built-in guides.

Ca' Foscari University of Venice

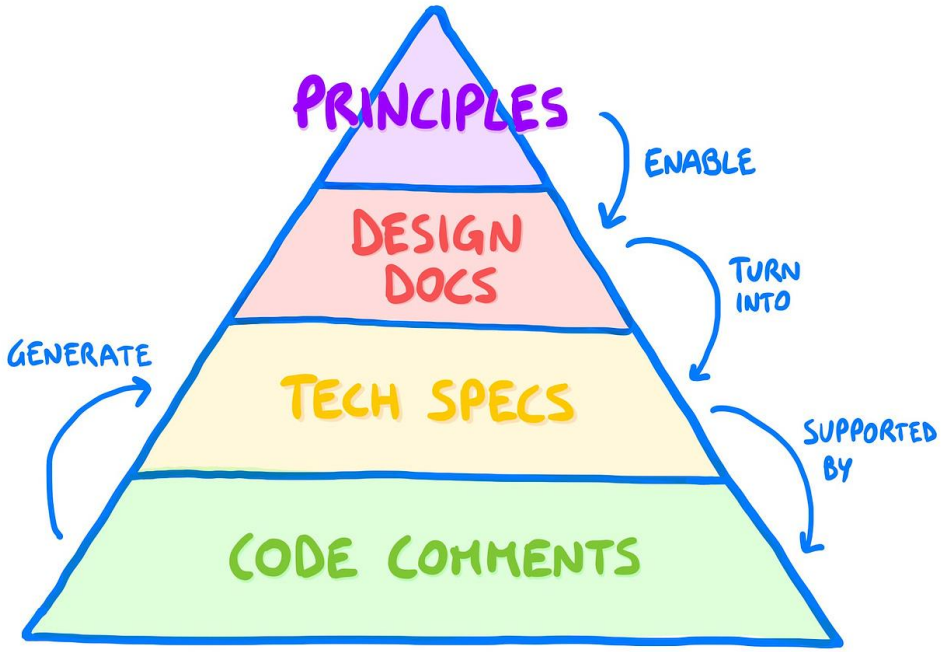
Type of documentation

- Project documentation
  - Requirements: Specifications of functional and non-functional requirements, describing what the software must do and the conditions it must meet.
  - Design documentation: illustrates the tech design and implementation strategy for a given initiative. It is meant to be created before you start writing code, and to be shared with other stakeholders to converge on the solution.
  - Decision logs: Notes on design choices, such as adopting specific technologies or implementing particular features, along with the reasoning behind them.
  - Test plans and reports: Document the tests performed on the software, including test cases and results. This is critical for ensuring software quality.
- Process documentation
  - Development procedures: Describes the methodologies used (e.g., Agile, Scrum) and work practices (e.g., code reviews, coding standards).
  - Deployment guides: Instructions on how to deploy the software to a production environment, including necessary configurations and steps for application updates.

Ca' Foscari University of Venice

Focus on main tech docs

- Principles — to describe how decisions should be made
- Design Docs — to describe how things are built, and record tech decisions
- Specs — to describe what's live, taking it from product requirement docs and design docs
- Code comments — to help maintainers and to keep track of small debt. In most languages you also have ways to auto-generate specs from comments.

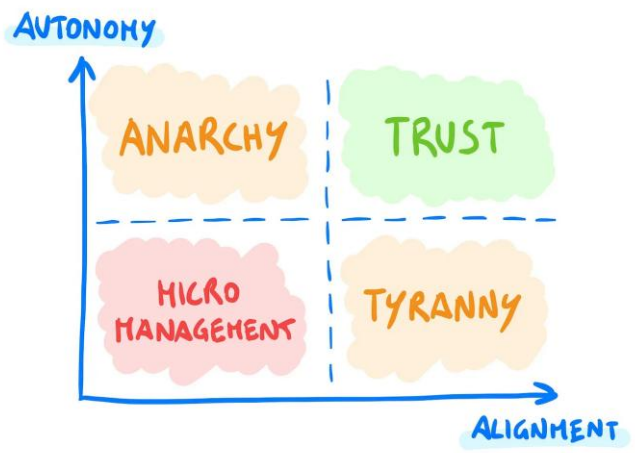


source: https://refactoring.fm/p/how-to-write-documentation

Principles, conventions and best practice

- Principles provide a blueprint for how decisions should be made.
- Without principles, you might meet goals in a way that does not reflect your culture.
  - deliver features without peer review
  - meet deadlines by making people overwork

- A set of shared beliefs that create alignement over how you do things:
  - a definition of what good looks like
  - a shared language to be used in daily work



source: https://refactoring.fm/p/how-to-write-documentation

Design Docs

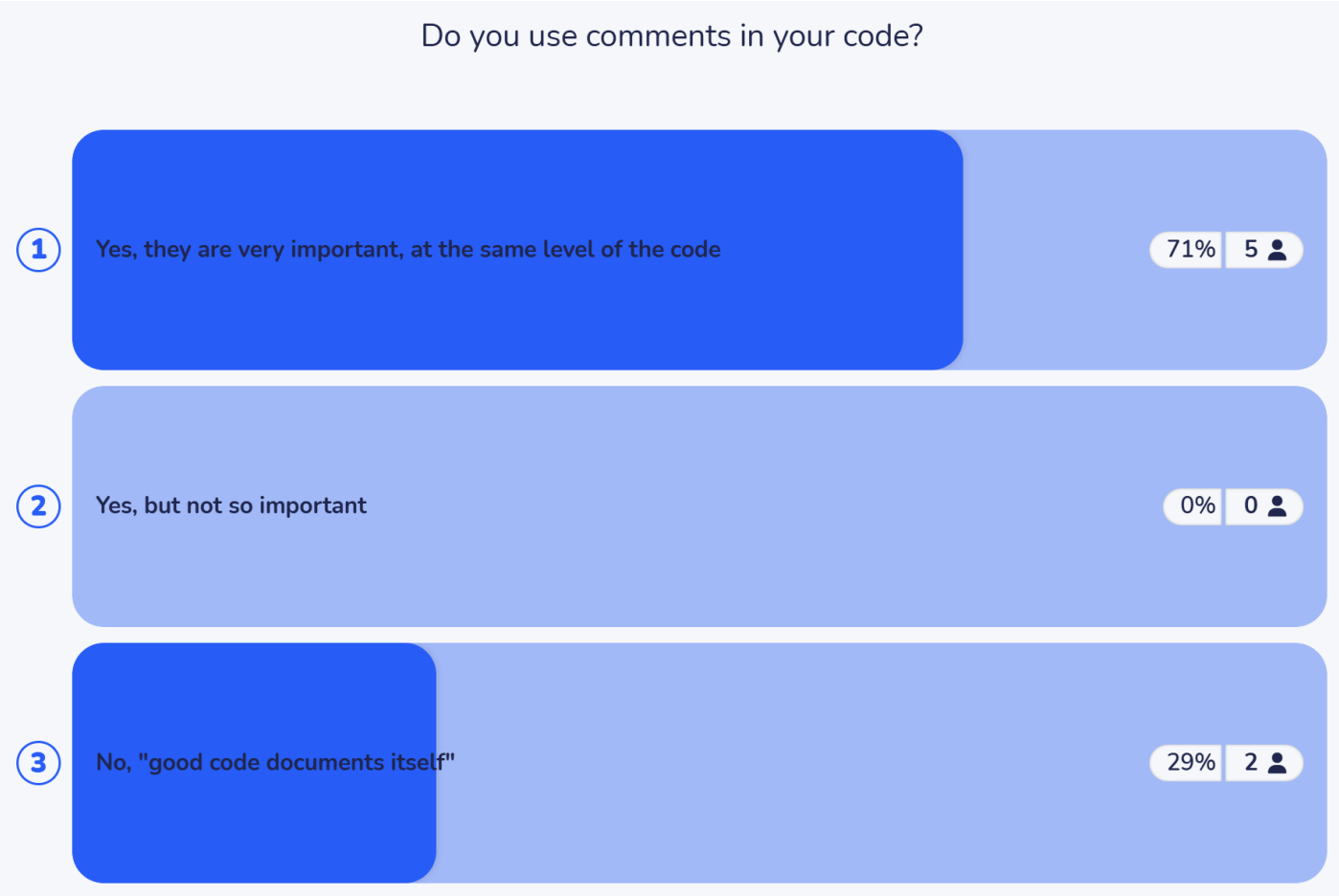
- Illustrates the tech design and implementation strategy for a given initiative.
- Created before you start writing code, and to be shared with other stakeholders to converge on the solution.
- Design docs are useful throughout the whole lifecycle of a project:
  - before release: they drive the process that makes people converge on a good solution
  - after release: they act as a decision record, which is often more useful than pure tech specs

Tech specs

- They act as a description of what is live.
- They may include database schemas, API specs and system diagrams.
- Updated after each update/change of the application.

Ca' Foscari University of Venice

Code comments



Code comments

- Commenting code is a somewhat controversial practice, as many developers believe that good code documents itself.
- My opinion: code comments are very important and permits to add value to the codebase (moreover help new team member to understand the application).
- Type of comments
  - comments at the top of the file/class: few lines to describe the file's primary goal can go a long way to help maintainers. It also helps to avoid scope creep over time and to keep the file true to its original goal
  - comments on complex functions: most functions should be simple enough to be understandable as-it-is, but there are algorithms and processes that cannot be so; in these cases, it helps to include a description of the inputs, the logic and the outputs of the functions
  - in-line comments: the need to comment single lines of code is most often a code smell; if people understand what the function does but not some of its lines, there is probably something you can rework, rather than commenting the lines themselves.

Ca' Foscari University of Venice

How to write docs

- Attach to processes — review principles on every performance review, create a design doc for every project, write comments whenever you write code. Turn writing docs into a recurring habit and support those habits with templates and checklists to make them easier.
- Organize into projects and areas — separate working docs for deliverables from long-term materials. Eventually turn the former into the latter.
- Archive old stuff — keep your workspace decluttered by hiding what is not useful anymore.

Ca' Foscari University of Venice

How to write docs

- Know your audience
  - understand who will read the documentation, tailor the language, depth, focus according to whether your readers are developers, end-users or stakeholders and adapt the content
- Organize your content clearly
  - create a clear structure using index, headings, subheadings; organizing information logically; follow a standard template for similar sections to create consistency
  - each element of lists must have a unique identifier
- Write in simple, concise language
  - avoid jargon when it is not necessary; use simple and clear language to make the documentation accessible
  - keep sentences short and straightforward to avoid confusion
  - be concise, focus on what the reader needs to know without adding unnecessary information
  - do not take anything for granted, write down everything

Ca' Foscari University of Venice

How to write docs

- Include examples and visuals
  - examples help readers understand how to use functions, endpoints
  - diagrams and screenshots can make complex processes or structures easier to understand
- Use a step-by-step approach guides
  - break down instructions into clear, sequential steps
  - test the instructions (go through the steps yourself or have a colleague do so to ensure that they are accurate and easy to follow)
- Keep the documentation up-to-date
  - regularly review and update
  - use tools to have a version control for your documentation (as Git do for your code)
  - mark outdated content clearly or archive it to avoid confusion
- Use tools and automation
  - leverage tools to streamline the creation and maintenance of documentation (e.g., Markdown, Docusaurus or Sphinx)
  - automate API documentation using tools like Swagger or Postman
  - integrate with CI/CD pipelines to ensure documentation gets updated along with software release
- Encourage Feedback
  - ask users for feedback and iterate based on feedback
- Be consistent with terminology and style
  - define a glossary and follow a defined style guide
- Focus on UX
  - make it easy to navigate through the documentation and include a "getting started" section that help

Ca' Foscari University of Venice

References and tools

- Some references
  - https://medium.com/free-code-camp/how-to-write-a-good-software-design-document-66fcf019569c
  - https://www.notion.so/blog/how-notion-uses-notion
  - https://fortelabs.com/blog/para/
- Tools
  - https://www.notion.so/
  - https://coda.io/
  - https://slite.com/
  - https://slab.com/
  - https://www.atlassian.com/software/confluence
  - https://www.onenote.com/
  - https://www.microsoft.com/it-it/microsoft-loop